



Sistema de cuentas bancarias

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

Tu dinero, en buenas manos



1 Enunciado del ejercicio

Desarrollar un programa en C++ y Java que permita registrar un cliente mediante su DNI y crear de una a tres cuentas bancarias. Para cada cuenta se registrará un número, un tipo (Ahorros o Corriente) y un saldo inicial no negativo. El usuario podrá seleccionar una cuenta y, desde un menú, ver sus atributos, enviar dinero, recibir dinero, hacer una transferencia entre sus propias cuentas, seleccionar otra cuenta o salir del sistema. Los montos de las operaciones deben ser positivos y no se permitirá enviar ni transferir una cantidad superior al saldo disponible.

2 Análisis de datos

1.1 Entrada de datos

- DNI del cliente: cadena de texto obligatoria y no vacía.
- Cantidad de cuentas: entero (1 a 3).
- Número de cada cuenta: cadena obligatoria y no vacía.
- Tipo de cuenta: 1 (Ahorros) o 2 (Corriente).
- Saldo inicial: real ≥ 0 .
- Cuenta seleccionada: entero (1 a n).
- Opción del menú: entero (1 a 6).
- Monto: real positivo.
- Cuenta destino: existente y diferente.

1.2 Proceso

- Registrar y validar el DNI.
- Solicitar y validar la cantidad de cuentas (máx. 3).
- Crear y validar cada cuenta (número, tipo y saldo).
- Seleccionar una cuenta existente.
- Repetir el menú hasta salir.
- Validar montos y cuentas en cada opción (enviar, recibir, transferir, cambiar).

1.3 Salida de datos

- Mensajes de validación.
- Atributos de la cuenta seleccionada.
- Confirmaciones y saldos actualizados.
- Mensajes de error y finalización.

3 Algoritmo

- Inicio.
- Declarar variables (cliente, cuentas, tipos, saldos, opciones y montos).
- Solicitar el DNI y validar que no esté vacío.
- Pedir la cantidad de cuentas (1 a 3).
- Para cada cuenta, solicitar y validar número, tipo y saldo inicial.
- Guardar la información en las variables de la cuenta 1, 2 o 3.
- Solicitar y validar la cuenta seleccionada.
- Cargar en variables actuales el número, tipo y saldo.
- Mostrar el menú y leer la opción.
- Ejecutar la opción según el menú (1 a 6).
- Repetir mientras la opción sea diferente de 6.
- Mostrar "Sistema finalizado".
- Fin.

4 Pseudocódigo

Algoritmo SistemaCuentasBancarias

Definir dni, numeroCuenta, tipoCuenta, numeroActual, tipoActual Como Cadena;
Definir cuenta1, cuenta2, cuenta3, tipo1, tipo2, tipo3 Como Cadena;
Definir cantidadCuentas, i, tipo, cuentaSeleccionada, destino, opcion Como Entero;
Definir saldoInicial, saldoActual, saldoDestino, monto, saldo1, saldo2, saldo3 Como Real;

Repetir

Escribir "Ingrese el DNI del cliente: "; Leer dni;
Hasta Que Longitud(dni) > 0;

Repetir

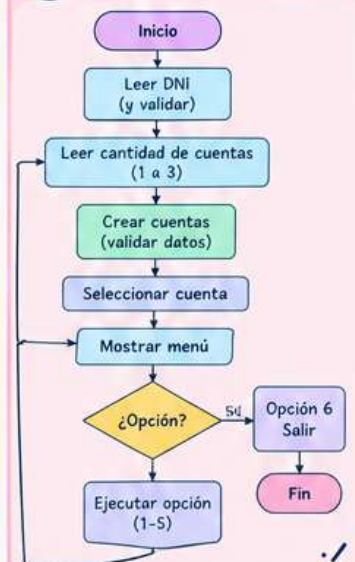
Escribir "¿Cuántas cuentas desea crear? (1 a 3): "; Leer cantidadCuentas;
Hasta Que cantidadCuentas >= 1 Y cantidadCuentas <= 3;
Para i <= 1 Hasta cantidadCuentas Hacer
// Validar número, tipo y saldo de cada cuenta
FinPara;
// ... (continuar con el menú y las opciones)

Repetir

Escribir "Seleccione una opción (1-6): "; Leer opcion;
Hasta Que opcion >= 1 Y opcion <= 6 Y opcion <> 6;
Escribir "Sistema finalizado.";

FinAlgoritmo

5 Diagrama de flujo



6 Prueba de escritorio

Caso 1: Cliente con 1 cuenta (Ahorros)

Variable	Valor
DNI	12345678
cantidad cuentas	1
número cuenta	001
tipo	1 (Ahorros)
saldo inicial	100.00

Operaciones:

- Ver atributos → OK
- Enviar 50 → saldo: 50.00
- Recibir 30 → saldo: 80.00
- Salir → finaliza

Caso 2: Transferencia entre cuentas

Variable	Valor
saldo cuenta 1	200.00
saldo cuenta 2	150.00
monto transferir	50.00
cuenta destino	2

Operaciones:

- Se valida que exista la cuenta destino.
- Se resta 50 a cuenta 1 (150.00).
- Se suma 50 a cuenta 2 (200.00).
- Mensaje de confirmación.

Caso 3: Datos inválidos

Dato inválido	Corrección
DNI vacío	→ se solicita de nuevo
Cantidad cuentas	5 → 3
Saldo inicial	-50 → 0 (o se rechaza)
Opción menú	9 → mensaje de error

Operaciones:

- Se muestran mensajes de validación.
- No se procesan datos incorrectos.
- El programa continúa normalmente.

7 Código

Java

```
import java.util.Scanner;

public class SistemaBancario {
    public static void main(String[] args) {
        // Aquí va el código en Java
    }
}
```

C++

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Aquí va el código en C++
    return 0;
}
```

8 Conclusiones

- El programa permite gestionar cuentas bancarias de forma segura y organizada.
- Se aplican correctamente las validaciones de datos y los límites establecidos.
- El uso de estructuras como while, if-else y switch facilita el control del flujo.
- El programa puede implementarse en C++ y Java sin mayores modificaciones.

9 Recomendaciones

- Mejorar la interfaz con un diseño más amigable.
- Agregar un archivo para guardar la información del cliente.
- Implementar más tipos de cuentas o funciones (consultas, historial, etc.).
- Usar clases y métodos para un código más limpio y ordenado.

Pequeños pasos,
grandes logros

